TASKFLOW

RAHUL KP KURUP

Task Management Application

Complete Technical Documentation

Stack: React • Node.js • Express • MongoDB • Docker

AI Assisted: Claude Web Free Version

Version 1.0 • April 2026

1. Project Overview

TaskFlow is a full-stack task management web application.
It is built using the MERN stack (MongoDB, Express, React, Node.js).
Docker is used to containerize the entire application.

1.1 Purpose

- Helps teams manage and track tasks efficiently.
- Supports task creation, assignment, and progress tracking.
- Provides a dashboard with real-time statistics.
- Allows admins to manage team members.
- Supports notifications for task assignments.

1.2 Key Highlights

Project Summary

Project Name: TaskFlow
Type: Full-Stack Web Application
Frontend: React (Vite) + Redux Toolkit + Tailwind CSS
Backend: Node.js + Express.js REST API
Database: MongoDB + Mongoose ODM
Auth: JWT (JSON Web Tokens) via HTTP-only cookies
Containerized: Docker + Docker Compose
AI Tool Used: Claude Web Free Version

1.3 Core Modules

- Authentication Module — Login, Register, Logout, JWT
- Task Module — Create, Update, Delete, Trash, Restore tasks
- User Module — Team management, profile updates
- Notification Module — Task assignment alerts
- Dashboard Module — Stats, charts, recent tasks

2. Tech Stack

Each technology is explained below with its purpose and location in the project.

2.1 React.js (Frontend UI Library)

- What it is: A JavaScript library for building user interfaces.
- Why used: Enables component-based, reusable UI development.
- Where used: All files inside client/src/ folder.
- Handles routing, pages, components, and state display.
- Used with Vite for fast development server and builds.

2.2 Redux Toolkit (State Management)

- What it is: A library for managing global application state.
- Why used: Shares auth state (user, theme) across all components.
- Where used: client/src/redux/ — store.js, slices/
- Uses RTK Query (apiSlice) for API calls to the backend.
- Handles login/logout, sidebar, and dark mode state.

2.3 Tailwind CSS (Styling Framework)

- What it is: A utility-first CSS framework.
- Why used: Rapid styling without writing custom CSS.
- Where used: All JSX component files in client/src/
- Supports dark mode via dark: prefix classes.
- Configured in client/tailwind.config.js

2.4 Node.js (Backend Runtime)

- What it is: JavaScript runtime environment for the server.
- Why used: Runs JavaScript on the server side.
- Where used: All files inside the server/ folder.
- Powers the Express.js API server.
- Uses ES Module syntax (import/export).

2.5 Express.js (Web Framework)

- What it is: Minimal Node.js web framework for REST APIs.
- Why used: Handles HTTP routing, middleware, and responses.

- Where used: server/index.js, server/routes/, server/middleware/
- All REST API endpoints are registered via Express Router.
- Middleware includes CORS, cookie-parser, morgan, and error handlers.

2.6 MongoDB (NoSQL Database)

- What it is: A document-oriented NoSQL database.
- Why used: Flexible schema for tasks, users, and notifications.
- Where used: Connected via Mongoose in server/utils/connectDB.js
- Stores Users, Tasks, and Notices as JSON-like documents.
- Runs as a Docker container (mongo:7.0 image).

2.7 Mongoose (ODM)

- What it is: Object Data Modeling library for MongoDB.
- Why used: Defines schemas, validation, and model methods.
- Where used: server/models/ — userModel.js, taskModel.js, notis.js
- Provides query helpers and pre-save password hashing hooks.
- Used to populate referenced documents (team members, task owners).

2.8 JSON Web Tokens — JWT (Authentication)

- What it is: A compact, URL-safe token standard for auth.
- Why used: Stateless authentication without server-side sessions.
- Where used: server/utils/index.js (createJWT), server/middleware/authMiddleware.js
- Token stored as HTTP-only cookie (secure, not accessible via JS).
- Expires in 1 day; verified on every protected request.

2.9 Docker & Docker Compose (Containerization)

- What it is: Platform for packaging and running applications in containers.
- Why used: Consistent environment across development and production.
- Where used: docker-compose.yml, server/Dockerfile, client/Dockerfile
- Three services: mongodb, server, client — all on a shared bridge network.
- Mongo data persisted in named volume (mongo_data).

2.10 Vite (Frontend Build Tool)

- What it is: A next-generation frontend tooling system.
- Why used: Extremely fast dev server with Hot Module Replacement.

- Where used: client/vite.config.js
- Proxies /api requests to the Express backend during development.
- Builds optimized production bundles.

3. System Architecture

TaskFlow follows a classic three-tier architecture.

3.1 Architecture Overview

Layer	Technology	Responsibility
Presentation (UI)	React + Redux + Tailwind	Renders UI, manages state, sends API requests
Application (API)	Node.js + Express.js	Business logic, routing, authentication
Data (DB)	MongoDB + Mongoose	Data storage, querying, schema validation
Containerization	Docker + Compose	Runs all 3 layers as isolated containers

3.2 Request-Response Flow

1. User interacts with the React UI in the browser.
2. React dispatches an RTK Query API call via apiSlice.
3. Vite dev proxy forwards the request to Express server on port 5001.
4. Express middleware validates the JWT cookie (protectRoute).
5. The matched route calls the appropriate controller function.
6. Controller queries MongoDB via Mongoose model.
7. MongoDB returns data to the controller.
8. Controller sends a JSON response back to React.
9. Redux store updates, React re-renders the UI.

3.3 Authentication Flow

10. User submits login form on the Login page.
11. React calls POST /api/user/login via authApiSlice.
12. Express userController.loginUser() checks credentials.
13. bcrypt compares the entered password with the hashed password.
14. On success, createJWT() sets an HTTP-only cookie with the JWT.
15. User data is returned; Redux authSlice stores it in localStorage.
16. All subsequent requests send the cookie automatically.

4. Project Structure Explanation

4.1 Root-Level Structure

File / Folder	Purpose
TaskFlow/	Root project directory
client/	React frontend application
server/	Node.js + Express backend API
docker-compose.yml	Defines all Docker services
.gitignore	Excludes node_modules, .env files from git
README.md	Project setup and usage guide

4.2 Server Folder Structure

Folder / File	Purpose
server/index.js	Application entry point. Sets up Express.
server/Dockerfile	Docker build instructions for the backend.
server/package.json	Node.js dependencies and scripts.
server/entrypoint.sh	Shell script run when container starts.
server/controllers/	Business logic for users and tasks.
server/models/	Mongoose schema definitions.
server/routes/	Express route definitions.
server/middleware/	Auth and error handling middleware.
server/utils/	Database connection and JWT helper.
server/seed/	Database seeding scripts and data.
server/tests/	Unit and integration test files.

4.3 Client Folder Structure

Folder / File	Purpose
client/src/App.jsx	Root component. Defines all routes.
client/src/main.jsx	ReactDOM entry point. Wraps app in Providers.
client/src/index.css	Global CSS and Tailwind directives.
client/src/pages/	Full-page view components (Dashboard, Tasks, etc.).
client/src/components/	Reusable UI components (Navbar, Sidebar, etc.).
client/src/redux/	Redux store, slices, and API definitions.
client/src/utils/	Utility functions and Firebase config.
client/src/tests/	Frontend component tests.
client/index.html	Root HTML file with app mount point.
client/vite.config.js	Vite configuration with API proxy.
client/tailwind.config.js	Tailwind CSS configuration.
client/Dockerfile	Docker build instructions for the frontend.

5. Module-Wise Explanation

5.1 Client Module

The client module is the React frontend application.

- Located in the client/ folder.
- Built with React 18 and Vite.
- Uses Redux Toolkit for global state management.
- Uses RTK Query for all backend API communication.
- Styled entirely with Tailwind CSS utility classes.
- Supports dark mode toggling via Redux state + Tailwind dark: classes.
- Routes are protected — unauthenticated users redirect to /log-in.

5.2 Server Module

The server module is the Express REST API backend.

- Located in the server/ folder.
- Exposes REST API endpoints under the /api prefix.
- Connects to MongoDB using Mongoose.
- Uses cookie-parser to read HTTP-only JWT cookies.
- Uses CORS middleware to allow only whitelisted origins.
- Logs HTTP requests in development using morgan.
- Has a /health endpoint for container health checks.

5.3 Auth Module

Handles all user identity and session management.

- Login: POST /api/user/login — validates credentials, sets JWT cookie.
- Register: POST /api/user/register — creates new user, hashes password.
- Logout: POST /api/user/logout — clears the JWT cookie.
- protectRoute middleware: verifies JWT on every protected endpoint.
- isAdminRoute middleware: restricts admin-only endpoints.
- Password is hashed with bcrypt (salt rounds: 12) before saving.

5.4 Task Module

Core module for creating and managing tasks.

- Create Task: POST /api/task/create — creates task, assigns team, sends notification.
- Get Tasks: GET /api/task — filters by stage, trash status, or search query.

- Update Task: PUT /api/task/:id — updates task fields.
- Trash Task: PUT /api/task/:id — sets isTrashed = true (soft delete).
- Delete/Restore: DELETE /api/task/:id — permanently deletes or restores.
- Subtasks: POST /api/task/:id/subtask — adds a subtask to a task.
- Activities: POST /api/task/activity/:id — logs activity on a task.
- Dashboard: GET /api/task/dashboard — returns grouped stats and recent tasks.

6. File-Level Explanation

6.1 Server Files

File	Purpose	Key Contents
server/index.js	Express app entry point	CORS, middleware, routes, server start
server/controllers/ taskController.js	Task CRUD logic	12 handler functions for all task operations
server/controllers/ userController.js	User CRUD logic	11 handler functions for auth and user management
server/models/userModel.js	User schema	Fields: name, email, password, isAdmin, tasks, isActive
server/models/taskModel.js	Task schema	Fields: title, stage, priority, team, activities, subTasks
server/models/notis.js	Notification schema	Fields: team, text, task, notiType, isRead
server/routes/index.js	Route aggregator	Mounts /user and /task routers
server/routes/userRoute.js	User API routes	All /api/user/* endpoint definitions
server/routes/taskRoute.js	Task API routes	All /api/task/* endpoint definitions
server/middleware/ authMiddleware.js	Auth middleware	protectRoute, isAdminRoute functions
server/middleware/ errorMiddleware.js	Error middleware	routeNotFound, errorHandler functions
server/utils/connectDB.js	DB connection	Mongoose connect with readyState guard
server/utils/index.js	JWT utility	createJWT() — signs and sets cookie
server/seed/seeder.js	DB seeder	Imports sample data into MongoDB
server/seed/data.js	Seed data	Sample users and tasks for development
server/tests/auth.test.js	Auth unit tests	Tests for user model and login endpoints
server/tests/tasks.test.js	Task integration tests	Tests for full task workflow
server/tests/setup.js	Test DB setup	MongoDB in-memory server for tests

6.2 Client Files

File	Purpose
client/src/App.jsx	Root component — defines all protected and public routes
client/src/main.jsx	ReactDOM.createRoot — wraps app in Redux Provider and Router
client/src/pages/Dashboard.jsx	Dashboard page — stats cards, charts, recent tasks table
client/src/pages/Tasks.jsx	Task list page — shows tasks filtered by stage
client/src/pages/TaskDetail.jsx	Single task detail — activities, subtasks, team members
client/src/pages/Login.jsx	Login form — email/password form with Redux auth dispatch
client/src/pages/Users.jsx	Users management page — admin can add/activate/deactivate users
client/src/pages/Trash.jsx	Trash page — shows soft-deleted tasks with restore/delete options
client/src/pages/Status.jsx	Status page — shows user-task status overview
client/src/components/Navbar.jsx	Top navigation bar — notifications, theme toggle, user menu
client/src/components/Sidebar.jsx	Left navigation — links to all main pages
client/src/components/Table.jsx	Reusable data table — used in Dashboard and Users pages
client/src/components/tasks/ AddTask.jsx	Add/Edit task modal form
client/src/components/tasks/ TaskCard.jsx	Task card component — used in board view
client/src/components/tasks/ BoardView.jsx	Kanban-style board view for tasks
client/src/components/tasks/ AddSubTask.jsx	Add subtask modal form
client/src/components/ NotificationPanel.jsx	Slide-in notification panel
client/src/components/ ConfirmationDialog.jsx	Reusable confirm/delete dialog
client/src/components/ ChangePassword.jsx	Change password modal form
client/src/components/AddUser.jsx	Add new team member modal form
client/src/components/SelectList.jsx	Reusable dropdown select component
client/src/components/ ModalWrapper.jsx	Generic modal wrapper with overlay

client/src/components/UserAvatar.jsx	Displays user initials as colored avatar
client/src/components/Button.jsx	Reusable styled button component
client/src/components/Textbox.jsx	Reusable text input with label
client/src/components/Loading.jsx	Animated loading spinner component
client/src/components/Tabs.jsx	Reusable tab navigation component
client/src/components/Title.jsx	Page title heading component
client/src/components/UserInfo.jsx	Displays user profile info card
client/src/redux/store.js	Redux store — combines apiSlice and authSlice
client/src/redux/slices/authSlice.js	Auth state — user, theme, sidebar open/close
client/src/redux/slices/apiSlice.js	Base RTK Query API slice
client/src/redux/slices/api/ authApiSlice.js	Auth API endpoints (login, logout, register)
client/src/redux/slices/api/ taskApiSlice.js	Task API endpoints (CRUD, dashboard, activities)
client/src/redux/slices/api/ userApiSlice.js	User API endpoints (team list, profile update)
client/src/utils/index.js	Utility helpers — date formatting, priority colors
client/src/utils/firebase.js	Firebase config (for file uploads if enabled)
client/src/tests/components.test.jsx	Frontend component unit tests
client/src/tests/setup.js	Vitest test environment setup

7. Function Breakdown

7.1 taskController.js

Function	Purpose
createTask()	Creates a new task, assigns team, sends notification
duplicateTask()	Copies an existing task with 'Duplicate - ' prefix
updateTask()	Updates task fields (title, date, priority, team, stage)
updateTaskStage()	Updates only the stage field of a task
updateSubTaskStage()	Marks a specific subtask as completed or incomplete
createSubTask()	Adds a new subtask to an existing task
getTasks()	Fetches tasks with optional filters (stage, trash, search)
getTask()	Fetches a single task with populated team and activities
postTaskActivity()	Appends a new activity log entry to a task
trashTask()	Soft-deletes a task by setting isTrashed = true
deleteRestoreTask()	Handles delete, deleteAll, restore, restoreAll actions
dashboardStatistics()	Returns total counts, grouped tasks, and graph data

7.2 userController.js

Function	Purpose
loginUser()	Validates credentials, creates JWT cookie, returns user data
registerUser()	Creates new user account with hashed password
logoutUser()	Clears the JWT cookie to end the session
getTeamList()	Returns all users with optional search filter
getNotificationsList()	Returns unread notifications for the logged-in user
getUserTaskStatus()	Returns all users with their assigned tasks populated
markNotificationRead()	Marks one or all notifications as read for the user
updateUserProfile()	Updates name, title, or role for user or admin target
activateUserProfile()	Toggles a user account active/inactive status
changeUserPassword()	Updates user password (re-hashed via pre-save hook)
deleteUserProfile()	Permanently deletes a user account from the database

7.3 authMiddleware.js

Function	Purpose
protectRoute()	Verifies JWT cookie — blocks unauthenticated access
isAdminRoute()	Checks req.user.isAdmin — blocks non-admin access

7.4 userModel.js (Mongoose Hooks)

Function	Purpose
pre('save') hook	Hashes password with bcrypt (salt: 12) before saving
matchPassword()	Compares entered password with stored hash using bcrypt.compare()

7.5 authSlice.js (Redux)

Action / Reducer	Purpose
setCredentials()	Saves user to Redux state and localStorage
logout()	Clears user from Redux state and localStorage
setOpenSidebar()	Opens or closes the mobile sidebar
toggleTheme()	Switches between light and dark theme
getUserFromStorage()	Reads persisted user from localStorage on app load
getThemeFromStorage()	Reads persisted theme from localStorage on app load

7.6 utils/index.js (Client)

Function	Purpose
formatDate()	Formats ISO date strings for display
getInitials()	Extracts user initials from full name
BGS[]	Array of Tailwind background color classes for avatars
PRIOTITYSTYELS{}	Maps priority levels to Tailwind color classes
TASK_TYPE{}	Maps task stages to Tailwind color classes

7.7 utils/index.js (Server)

Function	Purpose
createJWT()	Signs a JWT with userId and sets it as HTTP-only cookie (expires 1 day)

8. Features

8.1 Authentication Features

- User login with email and password.
- JWT stored in HTTP-only cookie (XSS-safe).
- User registration by admin.
- Secure logout by clearing cookie.
- Route protection — unauthenticated users redirected.
- Admin-only routes for privileged operations.

8.2 Task Management Features

- Create tasks with title, description, priority, due date, and team.
- Assign tasks to multiple team members.
- Update task details and stage at any time.
- Change task stage: Todo → In Progress → Completed.
- Duplicate an existing task with one click.
- Soft-delete tasks to the Trash bin.
- Restore or permanently delete trashed tasks.
- Add sub-tasks with title, date, and tag.
- Mark sub-tasks as completed or incomplete.
- Add file assets (URLs) and reference links to tasks.

8.3 Activity & Collaboration Features

- Log activity on any task (comment, started, bug, completed, etc.).
- View full activity history per task.
- Real-time notification sent on task assignment.
- Mark individual or all notifications as read.

8.4 Dashboard & Analytics

- Dashboard card showing total task count.
- Tasks grouped by stage (Todo, In Progress, Completed).
- Priority distribution bar chart.
- Table of last 10 recent tasks.
- Admin sees active user list with roles.

8.5 User Management (Admin Only)

- View all team members with search filter.
- Activate or deactivate user accounts.
- Update user name, title, and role.
- Delete user accounts permanently.
- Change own password via profile settings.

8.6 UI/UX Features

- Responsive design — works on mobile and desktop.
- Dark mode / Light mode toggle.
- Board view (Kanban) and list view for tasks.
- Slide-in notification panel in the Navbar.
- Confirmation dialogs before destructive actions.
- Toast notifications for success and error feedback.

9. Database Schema Design

9.1 User Schema

Collection: users | Model file: server/models/userModel.js

Field	Type	Required	Default	Notes
name	String	Yes	—	Full name of the user
title	String	Yes	—	Job title (e.g. Developer)
role	String	Yes	—	User role (e.g. Engineer)
email	String	Yes	—	Unique, lowercase, trimmed
password	String	Yes	—	Hashed with bcrypt (salt: 12)
isAdmin	Boolean	No	false	Grants admin privileges
tasks	[ObjectId]	No	[]	Ref to Task documents
isActive	Boolean	No	true	Account status flag
createdAt	Date	Auto	now()	Mongoose timestamp
updatedAt	Date	Auto	now()	Mongoose timestamp

9.2 Task Schema

Collection: tasks | Model file: server/models/taskModel.js

Field	Type	Required	Default	Notes
title	String	Yes	—	Task title
date	Date	No	Date.now	Task due date
priority	String	No	normal	Enum: high, medium, normal, low
stage	String	No	todo	Enum: todo, in progress, completed
activities	[Object]	No	[]	Array of activity log objects
activities.type	String	—	assigned	Enum: assigned, started, bug, etc.
activities.activity	String	—	—	Activity description text
activities.date	Date	—	now()	When activity was logged
activities.by	ObjectId	—	—	Ref to User who logged it
subTasks	[Object]	No	[]	Array of subtask objects

subTasks.title	String	Yes	—	Subtask title
subTasks.date	Date	No	—	Subtask due date
subTasks.tag	String	No	—	Subtask tag label
subTasks.isCompleted	Boolean	No	false	Completion flag
description	String	No	""	Task description text
assets	[String]	No	[]	Array of asset URLs
links	[String]	No	[]	Array of reference link URLs
team	[ObjectId]	No	[]	Ref to User documents
isTrashed	Boolean	No	false	Soft-delete flag

9.3 Notification (Notice) Schema

Collection: notices | Model file: server/models/notis.js

Field	Type	Required	Default	Notes
team	[ObjectId]	No	[]	Users who should receive this notice
text	String	Yes	—	Notification message content
task	ObjectId	No	—	Ref to the related Task document
notiType	String	No	alert	Enum: alert, message
isRead	[ObjectId]	No	[]	Users who have read this notice
createdAt	Date	Auto	now()	Mongoose timestamp
updatedAt	Date	Auto	now()	Mongoose timestamp

10. Docker Setup Guide

10.1 Prerequisites

- Install Docker Desktop: https://www.docker.com/products/docker-desktop
- Install Docker Compose (included with Docker Desktop).
- Clone the TaskFlow repository.

10.2 Environment Files

Create these environment files before running Docker:

server/.env

```
MONGODB_URI=mongodb://admin:adminpassword123@mongodb:27017/taskmanager?
authSource=admin
JWT_SECRET=your_super_secret_jwt_key_here
PORT=5000
NODE_ENV=development
CLIENT_ORIGIN=http://localhost:3000
```

client/.env

```
VITE_APP_BASE_URL=http://localhost:5001
```

10.3 Step-by-Step Docker Commands

17. Navigate to the project root folder:
    ```
    cd TaskFlow
    ```
18. Build and start all containers:
    ```
    docker-compose up --build
    ```
19. Run in detached (background) mode:
    ```
    docker-compose up --build -d
    ```
20. View running containers:
    ```
    docker ps
    ```
21. View server logs:
    ```
    docker logs tm_server
    ```
22. Stop all containers:
    ```
    docker-compose down
    ```
23. Stop and remove volumes (clears DB data):
    ```
    docker-compose down -v
    ```
24. Rebuild a single service:
    ```
    docker-compose up --build server
    ```
25. Access MongoDB shell inside container:

```
docker exec -it tm_mongodb mongosh -u admin -p adminpassword123
```

10.4 Docker Compose Service Explanations

mongodb service

- Image: mongo:7.0 (official MongoDB image).
- Container name: tm_mongodb.
- Exposes port 27017 on host.
- Sets root username and password via environment variables.
- Stores data in named volume mongo_data (survives restarts).
- Health check pings MongoDB every 10 seconds.
- Server service waits for this health check before starting.

server service

- Built from server/Dockerfile.
- Container name: tm_server.
- Depends on mongodb being healthy before starting.
- Exposes port 5001 on host (maps to internal port 5000).
- Reads environment variables from server/.env file.
- Mounts the server/ folder as a volume for live code changes.

network and volumes

- All three services share tm_network (bridge driver).
- Containers communicate using service names as hostnames.
- Example: server connects to mongodb via host mongodb:27017.
- mongo_data volume persists database data between restarts.

11. Testing

11.1 Backend Tests

Framework: Jest + Supertest + mongodb-memory-server

- Tests are located in: server/tests/
- Uses mongodb-memory-server for an in-memory test database.
- No real MongoDB connection required for tests.
- JWT_SECRET is set to a test-only value in each test file.

auth.test.js — Unit Tests

- User Model: Creates user with hashed password.
- User Model: matchPassword() returns true for correct password.
- User Model: matchPassword() returns false for wrong password.
- User Model: Rejects user creation without required fields.
- POST /api/user/login: Returns 200 and sets JWT cookie on success.
- POST /api/user/login: Returns 401 for wrong credentials.
- POST /api/user/login: Returns 400 for missing fields.

tasks.test.js — Integration Tests

- Admin can create a task (POST /api/task/create).
- Can fetch all tasks (GET /api/task).
- Can update a task stage (PUT /api/task/:id/stage).
- Can trash a task (PUT /api/task/:id).
- Can restore a trashed task.
- Non-admin only sees their own tasks.

11.2 Frontend Tests

Framework: Vitest + React Testing Library

- Test files located in: client/src/tests/
- setup.js configures jsdom and testing environment.
- components.test.jsx tests component rendering.

11.3 How to Run Tests

Backend tests:

```
cd server
npm install
npm test
```

Backend tests with coverage:

```
npm run test:coverage
```

Frontend tests:

```
cd client
npm install
npm test
```

Backend test setup (setup.js):

- connect() — starts in-memory MongoDB and connects Mongoose.
- closeDatabase() — drops the database and closes connection.
- clearDatabase() — clears all collections between tests.

12. Data Flow Explanation

12.1 Create Task — Full Data Flow

Step	Location	What Happens
1. User Action	React UI (AddTask.jsx)	User fills the Add Task form and clicks Submit.
2. API Call	taskApiSlice.js	RTK Query fires POST /api/task/create with form data.
3. Cookie Sent	Browser	JWT cookie is automatically attached to the request.
4. Middleware	authMiddleware.js	protectRoute() verifies the JWT. Sets req.user.
5. Route	taskRoute.js	POST /create route calls createTask controller.
6. Controller	taskController.js	createTask() validates, builds activity, calls Task.create().
7. Database	MongoDB	Task document is inserted. Notice document is created.
8. User Update	MongoDB	User.updateMany() pushes task._id to each team member.
9. Response	Express	Returns { status: true, task, message } with HTTP 201.
10. Redux Update	taskApiSlice.js	RTK Query cache is invalidated. Task list re-fetches.
11. UI Update	React	New task appears in the task list or dashboard.

12.2 Login — Full Data Flow

Step	Location	What Happens
1. User Action	Login.jsx	User enters email and password, submits the form.
2. API Call	authApiSlice.js	RTK Query fires POST /api/user/login with credentials.
3. Controller	userController.js	loginUser() finds user by email using User.findOne().
4. Password Check	userModel.js	matchPassword() compares entered password with bcrypt hash.
5. JWT Created	utils/index.js	createJWT() signs token with JWT_SECRET, sets HTTP-only cookie.
6. Response	Express	Returns user object (password removed) with HTTP 200.
7. Redux	authSlice.js	setCredentials() saves user to state and localStorage.
8. Redirect	App.jsx	Layout detects user in state, allows protected routes.

14. Conclusion

TaskFlow is a complete, production-ready task management application.
It demonstrates a well-structured MERN stack implementation.

14.1 What Was Achieved

- A fully functional REST API with 20+ endpoints.
- JWT-based authentication with HTTP-only cookies.
- Role-based access control (Admin vs Regular user).
- Real-time notifications on task assignments.
- A responsive React dashboard with dark mode support.
- Dockerized deployment with three independent services.
- Unit and integration test suites for backend and frontend.
- Database seeding scripts for rapid development setup.

14.2 Architecture Strengths

- Separation of concerns: controllers, models, routes are isolated.
- Stateless authentication scales horizontally without sessions.
- Docker Compose ensures consistent environments for all developers.
- RTK Query reduces boilerplate for API state management.
- MongoDB's flexible schema supports future feature expansion.

14.3 Potential Future Improvements

- Add real-time updates using WebSockets (Socket.io).
- Implement file upload to cloud storage (Firebase / S3).
- Add email notifications for task assignments.
- Introduce task commenting with @mentions.
- Add Gantt chart or calendar view for tasks.
- Add rate limiting and request throttling for security.
- Integrate CI/CD pipeline (GitHub Actions or similar).

14.4 AI Assistance

This project used Claude Web (Free Version) for AI assistance.

- AI helped generate boilerplate code and error handling patterns.
- Architecture decisions and design choices remain the developer's own.

- All code has been reviewed and tested by the RAHUL KP KURUP.

Summary

TaskFlow is a beginner-friendly yet production-grade MERN application.
It covers authentication, CRUD operations, notifications, and Docker.
This document serves as a complete technical reference for the project.